PREPRINT / AUTHOR MANUSCRIPT | A04 | VERSION 1.0 | 2026-08-07

The Recovery Graph: A Formal Model for Queryable Operational Continuity in Cloud-Native Systems

Obede Bessa Rocha da Silva

Independent Researcher - Clermont, Florida, United States | obedebessa@gmail.com | ORCID: 0009-0001-0787-5710

Abstract

This paper distinguishes infrastructure restoration from recovery of operating capability and introduces the Recovery Graph, a typed, attributed, evidence-carrying graph for representing recovery-specific dependencies, transitions, readiness, and confidence. It defines polynomial-time queries and six recovery metrics, maps the model to property-graph systems, and reports a bounded feasibility illustration. The results demonstrate model computability and queryability, not field effectiveness, causal impact, or production recovery performance.

Keywords

recovery graph; operational continuity; cloud-native systems; digital resilience; recovery readiness

Preprint status and posting policy

Unrefereed author version submitted to the International Journal of Computer Applications. This manuscript has not completed peer review.

IJCA permits unrefereed author preprints on free public preprint servers.

Planned repository: Zenodo. A permanent article DOI will be inserted after reservation and before public publication.

International Journal of Computer Applications (0975 - 8887)
*Volume * - No.*, August 2026*

The Recovery Graph: A Formal Model for Queryable Operational Continuity in Cloud-Native Systems

Obede Bessa
Independent Researcher
obedebessa@gmail.com

ABSTRACT

Cloud-native platforms have made the reconstruction of infrastructure routine: declarative configuration, GitOps pipelines, and reconciling controllers can re-create clusters, workloads, and network state with little human intervention. Yet public incident analyses repeatedly show that organisations whose infrastructure is fully *restored* remain unable to *recover*: services come back as running processes but not as operating capabilities, because the knowledge required to regain operational capability—recovery-specific dependencies, data-restoration order, credential re-establishment, capacity-gated ramp-up, and validation criteria—is tacit, unverified, and unqueryable. This paper formalises the distinction between *restore* (re-creating infrastructure) and *recover* (regaining operational capability) and introduces the *Recovery Graph*, a typed, attributed, evidence-carrying graph that makes operational recovery knowledge a first-class, machine-checkable artifact. We define the model's semantics—capability levels, recovery dependency edges, guarded recovery transitions, known-good evidence, readiness, and confidence—and give polynomial-time algorithms for recovery path computation, earliest-completion scheduling (yielding principled recovery time estimates), blocking-centrality analysis, circular-bootstrap detection, and counterfactual queries. Six metrics (recovery path length, recovery confidence score, recovery readiness index, recovery evidence completeness, recovery graph density, and recovery dependency coverage) turn recovery posture into a measurable, auditable quantity, aligning with emerging regulatory demands for demonstrable operational resilience. We show that the model maps directly onto standard property-graph query languages, present a reference architecture for populating and governing the graph, and report a computed feasibility illustration on a model of a widely used open-source microservice system, in which 75% of recovery dependencies are invisible to runtime observability. A registered evaluation design with four research questions, explicit threats to validity, and an open reference implementation completes the paper.

General Terms

Algorithms, Reliability, Performance

Keywords

operational continuity, disaster recovery, site reliability engineering, dependency graphs, cloud governance, digital resilience, microservices, graph queries

1. INTRODUCTION

Re-creating infrastructure has become a routine engineering activity. Declarative configuration, infrastructure-as-code, GitOps pipelines, and reconciling control planes such as Kubernetes allow an operator—or an automated controller—to rebuild clusters, redeploy workloads, and restore network and storage state from versioned descriptions with little manual effort [5, 11, 44]. When a region is lost or a cluster is corrupted, the machinery that answers "how do I get the infrastructure back?" is mature, automated, and continuously exercised, because the same machinery performs every ordinary deployment.

The machinery that answers "how do I get the *service* back?" is none of these things. Public incident reports repeatedly describe organisations whose infrastructure was restored—or was restorable at any moment—while the operational capability it was supposed to carry remained unavailable for hours. During the July 2024 CrowdStrike-related outage, remediation of each Windows host was mechanically trivial, yet fleets stalled because BitLocker recovery keys were stored in systems that were themselves down, and because the manual procedure had never been exercised at fleet scale [17, 43]. In Meta's October 2021 outage, the tooling and access systems needed to execute recovery depended on the network that had failed, forcing physical access to data centres [33]. GitLab's 2017 database incident became a canonical example of unverified recovery assets: of five backup and replication mechanisms believed to be in place, none had produced a usable, tested restore path when it was needed [25]. And Amazon's Kinesis event of November 2020 showed that even when infrastructure is healthy again, services may be unable to return to operation except through a slow, carefully ordered, capacity-aware restart [2]—a phenomenon since generalised as metastable failure [9, 29]. Systematic studies of cloud outages confirm that these are not isolated anecdotes: recovery, not detection, dominates outage duration, and recovery procedures are a recurring locus of failure [24, 26, 55].

Practitioners have long distinguished, informally, between bringing infrastructure back and bringing a service back; the distinction is implicit in ITIL's notion of service restoration [6], in disaster-recovery practice around recovery time objectives [50], and in the site reliability engineering literature [8]. What is missing is not the intuition but its formalisation: there exists, to our knowledge, no model in which the distinction is given precise semantics, no representation in which the knowledge required for operational recovery is stored as data rather than prose, and consequently no way to *query*, *measure*, or *audit* an organisation's ability to recover before

an incident forces the question. We name the two notions explicitly and use them throughout:

> **Restore** means re-creating infrastructure: resources exist, configuration is applied, processes run.
> **Recover** means regaining operational capability: the service verifiably does its job again, with its data, credentials, dependencies, capacity, and validation criteria satisfied.

Restoration is a prefix of recovery, not a synonym for it (Fig. 1). The gap between the two is where modern incidents are lost, and it is precisely the part of the problem for which today's platforms hold no machine-readable knowledge.
This paper asks whether that knowledge can be made a first-class artifact. Our research question is:

> *Can operational recovery be represented as a graph whose structure enables measurable and queryable recovery planning?*

As stated, the question is existential; to make it falsifiable we decompose it in Sec. 7 into four operational research questions concerning expressiveness (RQ1), divergence from runtime dependency structure (RQ2), predictive utility (RQ3), and queryability with governance value (RQ4).
Our answer is the *Recovery Graph*: a typed, attributed, evidence-carrying directed graph whose vertices are services and recovery resources, whose edges are *recovery dependency edges*—a relation that, as we show, is neither a subset nor a superset of the runtime call graph—and whose vertex state tracks discrete capability levels separating *restored* from *recovered*. The graph carries *known-good evidence*: timestamped, expiring records that a recovery action has been exercised successfully. From structure and evidence together we derive *recovery readiness*, *recovery confidence*, and computable *recovery paths* with principled recovery-time estimates.

Contributions.. This paper makes four contributions.

- **C1. A formal model** (Sec. 3) defining the service graph, the Recovery Graph, recovery dependency edges with a five-kind typology, capability levels and guarded recovery transitions, recovery evidence with freshness semantics, readiness, and confidence—together with well-formedness and consistency conditions and propositions characterising recoverability, schedule optimality, and the weakest-link behaviour of confidence.
- **C2. Algorithms and queryability** (Sec. 4): polynomial-time computation of recovery paths, earliest-completion schedules yielding $\mathrm{RTO}_{\mathrm{est}}$ estimates, blocking centrality, circular-bootstrap detection, and counterfactual removal; and a direct mapping of the model onto standard property-graph query languages [21, 32].
- **C3. A metric suite** (Sec. 5): recovery path length, recovery confidence score, recovery readiness index, recovery evidence completeness, recovery graph density, and recovery dependency coverage, with their formal properties, interpretation guidance, and failure modes—turning recovery posture into a quantity that can be tracked and audited, a need made concrete by regulation such as the EU's Digital Operational Resilience Act [20].
- **C4. A reference architecture, governance framing, and evaluation design** (Secs. 6–7): how the graph is populated from existing sources, kept consistent, and used as an auditable governance artifact; a registered-style experimental design with four research questions, baselines, and threats to validity; and an open reference implementation with a fully computed feasibility illustration on a model of a public microservice system.

Scope.. The Recovery Graph is a knowledge and planning model, not a data-replication protocol, a consensus mechanism, or an automated remediation engine: it does not make recovery actions correct, it makes recovery knowledge explicit, checkable, and queryable. We deliberately do not claim that a graph, by itself, shortens outages; whether graph-derived plans and estimates match observed recovery behaviour is an empirical question that Sec. 7 is designed to answer.
The remainder of the paper is organised as follows. Section 2 derives requirements from recurring recovery pathologies. Sections 3–5 present the model, algorithms, and metrics. Section 6 presents the reference architecture and governance mapping. Section 7 presents the evaluation design and a computed feasibility illustration. Section 8 positions the work; Sec. 9 discusses limitations; Sec. 10 concludes.

2. MOTIVATION AND PROBLEM ANALYSIS

We ground the model in recurring, documented failure patterns of operational recovery. We draw on primary post-incident reports published by the affected organisations and on systematic outage studies [24, 26, 41]; we return to the risk of anecdotal selection in Sec. 7.7.

2.1 Five recovery pathologies

P1: Recovery-only dependencies.. Recovering a service requires entities that the service never touches at runtime: container registries, infrastructure-as-code repositories, secret and key management systems, backup stores, identity providers, DNS control planes, out-of-band credentials. Because these dependencies are invisible to runtime observability—no trace ever crosses them—they are absent from every automatically discovered dependency graph, and their availability at recovery time is assumed rather than managed. The CrowdStrike remediation of July 2024 is a stark instance outside the cloud itself: the recovery procedure required BitLocker keys, and in multiple organisations the key-escrow systems were among the impacted machines [17, 43]. The 2021 OVHcloud datacentre fire showed the storage-layer variant: backups colocated with the primary site are a recovery dependency that fails jointly with what they protect [46].

P2: Circular bootstrap dependencies.. The transitive closure of recovery-only dependencies can contain cycles that only manifest under large-scale failure: the runbook wiki runs on the cluster the runbook rebuilds; the credential vault's unseal procedure is documented in the vault; the network-management tools reach devices over the network being repaired. Meta's October 2021 outage is the canonical public example—internal tooling and even physical-access systems depended on the failed network, materially slowing recovery [33]. Such cycles are invisible in steady state because they constrain only the *order of return*, not normal operation.

P3: Unverified recovery assets.. Backups that were never restored, failover paths never exercised, runbooks never executed since the architecture changed. GitLab's 2017 database incident documented five backup or replication mechanisms of which none yielded a working restore when needed [25]. Industry analyses repeatedly find that recovery procedures fail at first use [1, 55]; Google institutionalised disaster-recovery testing precisely because

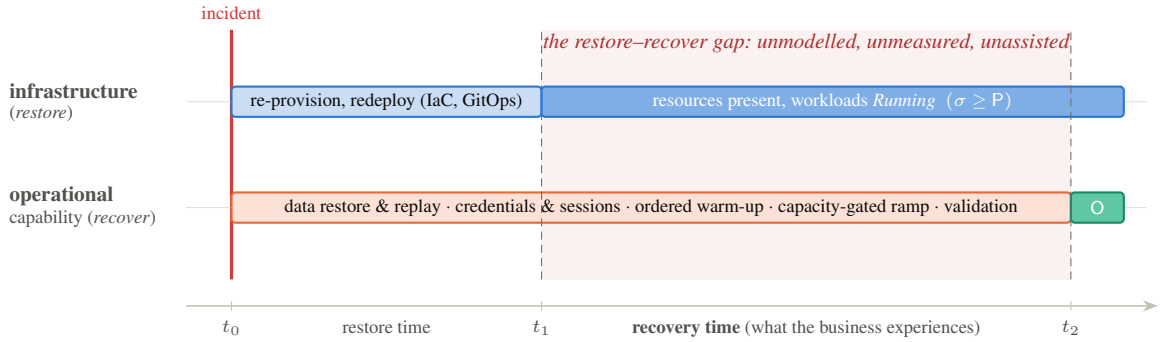

Fig. 1. The restore–recover gap. Cloud-native tooling automates and measures the left interval; the right interval—where data restoration, credential re-establishment, dependency-ordered warm-up, capacity-gated ramp-up, and validation occur—is typically governed by prose runbooks and tacit knowledge. The formal capability levels P (provisioned) and O (operational) are defined in Sec. 3.

untested recovery knowledge decays silently [39]. The epistemic point is general: *the ability to recover is a claim that expires*, yet no common representation records when each such claim was last substantiated.

P4: Unordered, capacity-blind restart.. When many services return simultaneously, retry storms, cold caches, and thundering-herd effects can hold a fully restored system in a degraded state—the metastable failure pattern [9, 29]. AWS's Kinesis event of November 2020 required a deliberately slow, ordered restart of front-end fleets to avoid re-entering failure [2]; the December 2021 us-east-1 event similarly involved congestive collapse of an internal network during recovery attempts [3]. Recovery order and ramp rate are first-class constraints, but nowhere are they represented as data that a planner could consume.

P5: Recovery knowledge as prose.. Where recovery knowledge exists, it lives in runbooks and wikis: unstructured, unverifiable, quickly stale, and unqueryable. Empirical studies of production incidents at a large cloud provider report outdated or incorrect troubleshooting documentation as a recurring aggravator [14, 24]; the SRE literature acknowledges the maintenance burden of playbooks even as it recommends them [8]. Prose cannot answer "what must be recovered before service X?", "which tier-1 services have no fresh restore evidence?", or "what is our best-case recovery time for checkout?"—questions that are answerable in seconds from a suitable data structure.

2.2 Why existing layers do not hold this knowledge

Each layer of the cloud-native stack maintains a model of the system, and each model stops short of operational recovery. Declarative infrastructure and GitOps hold the *desired infrastructure state* and can converge a cluster toward it [5, 44]; the resource graphs of tools such as Terraform order the creation of *resources*, not the return of *capabilities* [27]. Distributed tracing yields the *runtime call graph* [42, 49], which, as Sec. 3 makes precise, is neither a subset nor a superset of the recovery dependency relation. Configuration management databases hold *asset inventory and relationships* but no recovery semantics, evidence, or executable interpretation, and are notoriously stale [6]. Service catalogues record ownership metadata [15]. Monitoring records *current* health, not the preconditions of regaining health. Chaos engineering and DR drills *generate* exactly the evidence that would substantiate recovery claims [7, 39, 47]—but that evidence is filed in reports and tickets, not attached to the entities it substantiates. Table 1 traces each pathology to the knowledge that no existing layer represents and to the requirement it induces.

Two clarifications bound the argument. First, we do not claim that organisations are ignorant of these pathologies—post-incident reviews name them routinely; we claim that the knowledge that would prevent them is not *represented* anywhere that a machine, an auditor, or an incident commander can query. Second, the pathologies are drawn from public reports of large operators because only those are citable; the evaluation design (Sec. 7) therefore includes modelling studies on independent systems to test whether the pathologies, and the model's coverage of them, generalise.

3. THE RECOVERY GRAPH MODEL

This section formalises the model against requirements R1–R6 (Table 1). Throughout, fix a system under management comprising a finite set of *services* S (independently deployable units of capability) and a finite set of *recovery resources* R (entities required to execute or validate recovery but not themselves part of the served workload: registries, code and configuration repositories, secret stores, backup stores, DNS and identity control planes, observability stacks, external sandboxes). We write $V = S \cup R$.

3.1 Preliminaries: the service graph

DEFINITION 1 SERVICE GRAPH. *The* service graph *is a directed graph $G_S = (S, D)$ with $D \subseteq S \times S$, where $(u, v) \in D$ iff u invokes or consumes v during normal operation. G_S is the object produced by distributed tracing and topology discovery [42, 49].*

G_S describes how work flows when the system is healthy. Nothing in G_S describes how capability returns when it is not; establishing the relationship between the two graphs precisely is the task of Sec. 3.3.

3.2 Capability levels: restore vs. recover, formally

DEFINITION 2 CAPABILITY LEVELS. *Let $\mathcal{L} = \{\mathsf{D}, \mathsf{P}, \mathsf{F}, \mathsf{O}\}$ be totally ordered by $\mathsf{D} < \mathsf{P} < \mathsf{F} < \mathsf{O}$, with the reading:* D *(*down*) — the node is unavailable;* P *(*provisioned*) — the node's infrastructure exists and its processes run;* F *(*functional*) — the node passes its own health and self-checks and can serve, possibly in degraded mode;* O *(*operational*) — the node satisfies its declared validation predicate val_v (synthetic end-to-end transactions, SLO-*

Table 1. From observed pathologies to model requirements.

Pathology	**Missing machine-readable knowledge**	**Requirement**
P1 recovery-only deps	entities needed only at recovery time, per service	**R1** model recovery-specific dependencies over services *and* non-runtime resources
P2 circular bootstrap	order-of-return constraints and their cycles	**R2** make dependency structure analysable (cycles, seeds, order) before incidents
P3 unverified assets	when each recovery claim was last substantiated	**R3** attach expiring, verifiable evidence to recovery capabilities
P4 capacity-blind restart	ordering, gating, and ramp constraints between services	**R4** distinguish capability levels and gate transitions on dependency state
P5 prose knowledge	any queryable representation at all	**R5** support ad hoc queries and computed plans/estimates
governance (cross-cutting)	auditable posture over time	**R6** yield metrics that are monotone in evidence and coverage, suitable for audit

conformant probes, data-consistency checks). A recovery state *is a map* $\sigma : V \to \mathcal{L}$.

DEFINITION 3 RESTORE, RECOVER. *A node* v *is* restored *in state* σ *iff* $\sigma(v) \geq \mathsf{P}$*, and* recovered *iff* $\sigma(v) = \mathsf{O}$. *A target set* $T \subseteq S$ *is recovered iff all its members are.*

Two design choices deserve defence. First, O is defined relative to a *declared* validation predicate, not to an absolute notion of correctness: this is what makes "recovered" auditable—an organisation states in advance what evidence of operation it will accept, and the model checks that statement, a discipline analogous to declaring SLOs [8]. Second, $\mathcal{L}$ is deliberately a small total order. Real systems admit richer partial orders of degraded operation; we discuss the extension in Sec. 9 and note here only that every mechanism below is parametric in $\mathcal{L}$ and requires only that it be a finite lattice. Figure 2 shows the levels and the guarded transitions between them defined in Sec. 3.5: *restore* is the prefix $\mathsf{D} \to \mathsf{P}$; *recover* is the whole word $\mathsf{D} \to \mathsf{O}$.

3.3 Recovery dependency edges

DEFINITION 4 RECOVERY DEPENDENCY EDGE. *A* recovery dependency edge *is a tuple* $e = (u, v, \kappa, h, \theta, \pi)$ *with* $u, v \in V$*, kind* $\kappa \in K = \{\mathsf{order}, \mathsf{data}, \mathsf{verify}, \mathsf{capacity}, \mathsf{authz}\}$*, hardness* $h \in \{\mathit{hard}, \mathit{soft}\}$*, stage* $\theta \in \{\mathit{start}, \mathit{complete}\}$*, and threshold* $\pi \in \mathcal{L}$*. We read* e *as "*u requires v*": if* $h = \mathit{hard}$ *and* $\theta = \mathit{start}$*, no recovery action of* u *may begin until* $\sigma(v) \geq \pi$*; if* $h = \mathit{hard}$ *and* $\theta = \mathit{complete}$*,* u *may progress through intermediate levels but cannot be certified* O *until* $\sigma(v) \geq \pi$*. Soft edges never block; they record dependencies under which* u *can proceed in degraded form (*degraded-start*), and they discount confidence (Def. 12).*

The central structural claim of the paper is that this relation is genuinely different from the runtime relation:

PROPOSITION 1 INCOMPARABILITY. *There exist systems for which, writing*

$$E_{\mathrm{R}}^{\flat} = \{(u, v) : (u, v, \ldots) \in E_{\mathrm{R}}\},$$

for the underlying edge pairs, neither $E_{\mathrm{R}}^{\flat} \subseteq D$ *nor* $D \subseteq E_{\mathrm{R}}^{\flat}$ *holds.*

PROOF (BY CONSTRUCTION). (i) $E_{\mathrm{R}}^{\flat} \not\subseteq D$: any service pulled from a container registry has a hard order recovery dependency on the registry, yet no runtime call ever reaches the registry, so the pair is absent from D; backup stores, IaC repositories, and key escrow (Sec. 2.1, P1) are analogous. (ii) $D \not\subseteq E_{\mathrm{R}}^{\flat}$: a service that calls an advertising sidecar at runtime but starts and validates without it yields $(u, \mathit{ad}) \in D$ classified *irrelevant* or *soft*, hence absent from the hard recovery relation. □

The proposition is deliberately modest—an existence claim, provable by exhibition. The empirically interesting quantity is the *magnitude* of the divergence in real systems, which we operationalise as the recovery-only ratio ρ and study under RQ2 (Sec. 7); in the worked example of Sec. 7.6, three quarters of recovery dependencies have no runtime counterpart. Figure 3 contrasts the two graphs on a slice of that example.

3.4 The Recovery Graph

DEFINITION 5 RECOVERY GRAPH. *A* Recovery Graph *is a structure* $G_{\mathrm{R}} = (V, E_{\mathrm{R}}, \tau, B, \mathit{req}, \mathit{Ev}, \mu)$ *where* $V = S \cup R$*;* E_{R} *is a finite set of recovery dependency edges over* V *(Def. 4);* $\tau : V \to \{\mathsf{svc}, \mathsf{res}\}$ *types the vertices;* $B \subseteq V$ *is the* seed set*—nodes recoverable by means external to the modelled system (managed services, out-of-band procedures);* $\mathit{req} : V \to 2^{\mathcal{C}}$ *assigns required evidence classes (Sec. 3.6);* Ev *is the evidence store; and* μ *carries metadata: criticality weights* $w_v > 0$*, recovery targets, per-level action durations* $d_v(\ell)$*, and the* classification $\gamma : D \to \{\mathit{hard}, \mathit{soft}, \mathit{irrelevant}\}$ *recording, for each runtime edge, its assessed recovery relevance.*

The classification γ makes the relationship between G_{S} and G_{R} an explicit, auditable artifact rather than an implicit judgment: a runtime edge may induce a hard recovery edge, a soft one, or none, but under consistency rule C2 (Sec. 3.8) it may not be silently ignored. This is the basis of the coverage metric RDC (Sec. 5).

Well-formedness.. G_{R} is *well-formed* iff: (**W1**) every non-seed node has a defined action for each level transition (executability); (**W2**) all edges reference nodes of V and all thresholds satisfy $\pi \geq \mathsf{P}$; (**W3**) the hard core is acyclic at action granularity (Def. 7); (**W4**) every $\preceq$-minimal node of the hard enablement order is a seed. W3 and W4 are exactly the absence of pathology P2: a violation is a circular bootstrap or an unseeded origin, detected before an incident rather than during one.

3.5 Actions, transitions, and recovery plans

DEFINITION 6 RECOVERY ACTION. *For each* $v \in V$ *and level* $\ell \in \{\mathsf{P}, \mathsf{F}, \mathsf{O}\}$*, the* recovery action a_v^{ℓ} *advances* v *from the pre-*

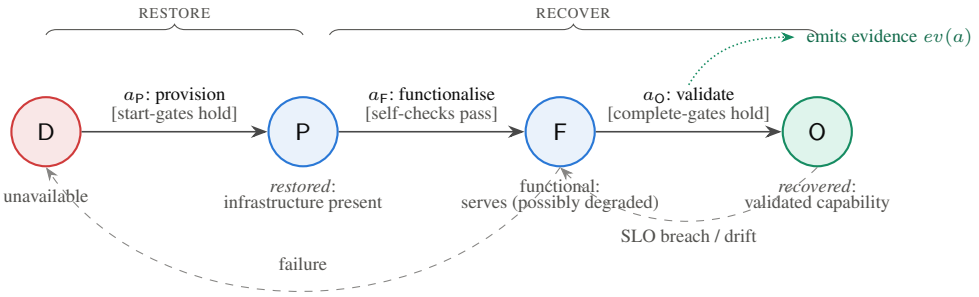

Fig. 2. Capability levels and guarded recovery transitions (Defs. 2, 6). Each transition is an action whose guards refer to the levels of other nodes via recovery dependency edges; completing a transition may emit evidence (Def. 10). Dashed edges are failure/degradation events, which are outside the planned-recovery semantics (Assumption A1).

Table 2. Edge-kind typology with recovery semantics and requirement coverage.

Kind	**Semantics and typical instance**	**Req.**
order	v must be sufficiently capable for u's recovery *actions* to execute: image pulls need the registry, cluster rebuild needs the IaC repository	R1, R2
data	u's state derives from v: restore-before-serve, replica rebuild, log replay; may carry a staleness bound (the edge-level analogue of RPO)	R1
verify	u's validation (O) requires v: synthetic checkout needs the payment sandbox; every probe needs the observability stack	R4
capacity	v must have headroom before u ramps traffic: the anti-metastability gate [9]	R4
authz	credentials or approvals for u's recovery are obtained from v: key management, identity provider, change approval	R1

decessor level of ℓ to ℓ. It has duration $d_v(\ell) \geq 0$ and may emit evidence $ev(a_v^\ell) \subseteq \mathcal{C}$. Its guard is the conjunction of (i) v occupying the predecessor level, (ii) for every hard start-edge $(v, x, \kappa, hard, start, \pi)$: $\sigma(x) \geq \pi$ when $\ell = \mathsf{P}$, and (iii) for every hard complete-edge $(v, x, \kappa, hard, complete, \pi)$: $\sigma(x) \geq \pi$ when $\ell = \mathsf{O}$.

DEFINITION 7 ACTION GRAPH. *The* action graph $\mathcal{A}(G_{\mathrm{R}})$ *has one task node (v, ℓ) per action, weighted $d_v(\ell)$; intra-node edges $(v, \mathsf{P}) \to (v, \mathsf{F}) \to (v, \mathsf{O})$; and, for each hard edge of E_{R}, a cross edge $(x, \pi) \to (v, \mathsf{P})$ if $\theta = start$ and $(x, \pi) \to (v, \mathsf{O})$ if $\theta = complete$.*

Working at action granularity matters: a pair of nodes may depend on each other at *different levels* without deadlock (the observability stack needs the cluster at F; the cluster's O-validation needs observability), and $\mathcal{A}(G_{\mathrm{R}})$ distinguishes such benign mutual references from true cycles. This resolves an apparent paradox in several public incidents: what looks like a circular dependency is sometimes recoverable through interleaving at intermediate levels, and the model decides which case one is in.

DEFINITION 8 RUN, PLAN, SCHEDULE. *Under* Assumption A1 *(monotone runs: during planned recovery no node regresses in level; a regression event ends the run and re-plans), a* recovery run *from initial state σ_0 is a sequence of action firings each enabled by its guard. A* recovery plan *for target $T \subseteq S$ is a run after which $\sigma(v) = \mathsf{O}$ for all $v \in T$. A* schedule *assigns start times respecting guards and durations; its* makespan *for v is the completion time of (v, O), and $\mathrm{RTO}_{\mathrm{est}}(v)$ denotes the minimum such makespan over schedules with unbounded parallelism.*

THEOREM 9 MODELLED RECOVERABILITY. *Let G_{R} satisfy W1–W2, and let $\sigma_0 \equiv \mathsf{D}$ (total loss). Every $v \in V$ is recoverable—some plan brings it to* O*—iff (i) $\mathcal{A}(G_{\mathrm{R}})$ is acyclic and (ii) every source task of $\mathcal{A}(G_{\mathrm{R}})$ belongs to a seed (W3–W4).*

PROOF SKETCH. ($\Leftarrow$) Fire actions in any topological order of $\mathcal{A}(G_{\mathrm{R}})$; guards hold inductively because all guard atoms are level lower-bounds established by earlier tasks and never invalidated under A1; seeds discharge base cases. ($\Rightarrow$) A cycle in $\mathcal{A}(G_{\mathrm{R}})$ yields a set of tasks none of which can fire first (circular waiting); a non-seed source has a guard no run can establish. Full proof in A. □

We stress the qualifier *modelled*: the theorem certifies the declared knowledge, not the world; an unmodelled dependency can defeat a certified plan. This is not a weakness peculiar to the model—it is the condition of every plan—but the model makes the exposure measurable (via RDC and drift, Secs. 5, 3.8) instead of tacit.

PROPOSITION 2 SCHEDULES AND $\mathrm{RTO}_{\mathrm{est}}$. *For well-formed G_{R} with deterministic durations and unbounded executors, the earliest-completion schedule and $\mathrm{RTO}_{\mathrm{est}}(v)$ for all v are computable in $O(|V_{\mathcal{A}}| + |E_{\mathcal{A}}|)$ time by longest-path propagation over $\mathcal{A}(G_{\mathrm{R}})$ in topological order—the classic critical-path computation [37]. With a bound k on concurrent actions, minimising makespan is NP-hard (it contains $P \,|\, prec \,|\, C_{\max}$ [54]), and the unbounded value remains a valid lower bound.*

COROLLARY 1 OPTIMISM OF OMISSION. $\mathrm{RTO}_{\mathrm{est}}$ *is monotone non-decreasing under addition of hard edges and non-negative durations. Hence an* under-modelled *graph can only make $\mathrm{RTO}_{\mathrm{est}}$ optimistic, never pessimistic.*

Corollary 1 has a governance consequence we exploit in Sec. 5: a recovery-time estimate is only as trustworthy as the completeness of the graph behind it, so the estimate must always be reported together with the coverage metric RDC; the two are designed as a pair.

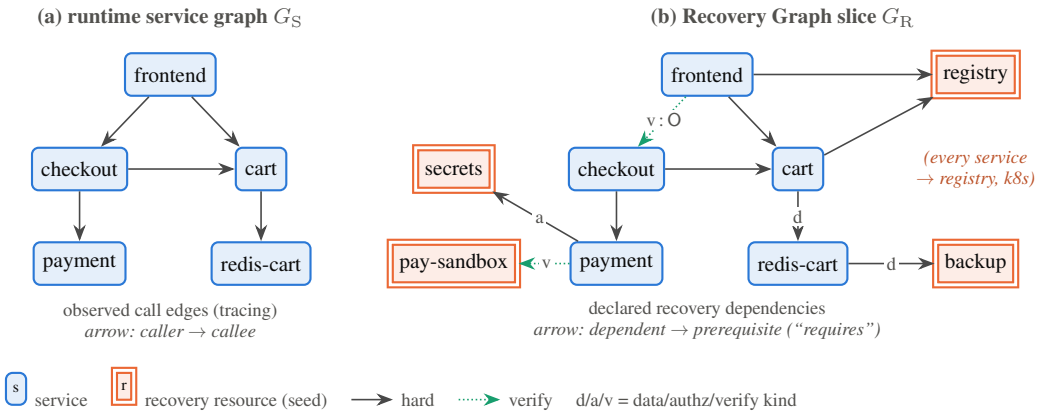

Fig. 3. The same subsystem seen by tracing (a) and by the Recovery Graph (b). Recovery adds nodes invisible at runtime (registry, backup, secrets, payment sandbox), types and gates the edges, and demotes runtime edges that are not recovery-relevant. Neither graph is derivable from the other (Prop. 1).

3.6 Recovery evidence and known-good evidence

Requirement R3 demands that the *claims* embodied in the graph—this backup restores, this runbook works, this failover engages—be substantiated and that the substantiation expire. Let $\mathcal{C}$ be a finite set of *evidence classes*; Table 3 lists the taxonomy used in this paper (organisations may refine it).

DEFINITION 10 RECOVERY EVIDENCE; KNOWN-GOOD. *An* evidence item *is a tuple* $e = (v, c, t, \tau_e, verdict, prov)$*: subject* $v \in V$*, class* $c \in \mathcal{C}$*, production time* t*, validity horizon* $\tau_e > 0$*,* $verdict \in \{pass, fail\}$*, and provenance* $prov$ *(who or what produced it, linking to raw records). At time* t_{now}*,* e *is* known-good *iff* $verdict = pass$ *and* $t_{\text{now}} - t \le \tau_e$*. Its* freshness *is* $\varphi(e) = \max\big(0,\, 1 - (t_{\text{now}} - t)/\tau_e\big)$.

The linear decay is a default, not a dogma: any non-increasing φ with $\varphi = 1$ at production and $\varphi = 0$ at expiry is admissible, and the metric definitions below are parametric in it; the evaluation design includes a sensitivity analysis over decay shapes (Sec. 7.4). What is *not* negotiable is expiry itself: Definition 10 is the formal rendering of the observation in Sec. 2.1 that the ability to recover is a claim that expires. A backup integrity check from last week supports a claim; one from last year supports an assumption.

3.7 Readiness and confidence

DEFINITION 11 RECOVERY READINESS. *Node* v *is* ready, *written* $ready(v)$*, iff for every required class* $c \in req(v)$ *there exists a known-good evidence item for* (v, c)*. Readiness is local and binary by design—it is the auditable atom.* Transitive readiness $ready^*(v)$ *additionally requires* $ready(x)$ *for every* x *in the hard closure* $\mathrm{cl}(v)$ *(Def. 13).*

DEFINITION 12 RECOVERY CONFIDENCE. *Let* $\mathrm{pre_h}(v)$ *and* $\mathrm{pre_s}(v)$ *be* v*'s hard and soft prerequisites. The* local evidence score *is*

$$E_{\text{loc}}(v) = \frac{1}{|req(v)|} \sum_{c \,\in\, req(v)} \max_{\substack{e \in Ev(v,c) \\ verdict(e) = pass}} \varphi(e),$$

with $E_{\text{loc}}(v) = 1$ *if* $req(v) = \emptyset$*, and* recovery confidence *is defined, on graphs whose hard core is acyclic, by the recursion (evaluated in topological order)*

$$C(v) \;=\; E_{\text{loc}}(v) \cdot \min_{x \,\in\, \mathrm{pre_h}(v)} C(x) \cdot \prod_{x \,\in\, \mathrm{pre_s}(v)} \big(1 - \omega_s\,(1 - C(x))\big),$$

with $\min \emptyset = 1$*,* $\prod \emptyset = 1$*, and soft-discount parameter* $\omega_s \in [0, 1]$ *(default* $\tfrac{1}{2}$*).*

PROPOSITION 3 WEAKEST LINK. *For well-formed* G_R*:* $0 \le C(v) \le 1$*;* $C(v) \le C(x)$ *for every hard prerequisite* x *of* v*, hence* $C(v) \le \min_{x \in \mathrm{cl}(v)} C(x)$*; and* C *is monotone non-decreasing in the addition of passing evidence and non-increasing in the passage of time. (Proof in A.)*

The min over hard prerequisites is a modelling commitment, not an estimate: hard recovery is conjunctive, and a chain is as credible as its least-substantiated link. The multiplicative local factor further penalises nodes that are themselves poorly evidenced. Two consequences are intended. First, confidence localises: the *argmin* witness on each node's chain identifies the confidence bottleneck, a directly actionable query (Sec. 4). Second, a single decayed credential or stale rehearsal at depth collapses the confidence of everything above it—which is not an artifact but the model faithfully reporting P1/P3 exposure, as the worked example demonstrates (Sec. 7.6).

REMARK 1 CONFIDENCE IS NOT PROBABILITY. *C is an evidence-weighted index in $[0, 1]$, suitable for ranking, trending, thresholding, and bottleneck localisation. It is* not *a calibrated probability of successful recovery: no independence assumptions are made and none would be defensible. Calibrating C against observed drill outcomes is an explicit direction of the evaluation design (RQ3) and of future work, in the spirit of dependency-aware availability estimation [53].*

3.8 Consistency

A Recovery Graph is a claim about the world and must be checked against it continuously. We define six mechanisable *consistency rules*: (**C1**) *node coverage*: every deployed service discovered by inventory or tracing has a node in V; (**C2**) *classification totality*: γ is total on D—no runtime edge is unassessed; (**C3**) *referential executability*: every action's referenced assets resolve (runbook URLs live, image digests present, IaC references pinned); (**C4**) *ev-*

Table 3. Evidence classes $\mathcal{C}$ with typical producers.

Class	Substantiates	Typical producer
BI backup integrity	data edges: the artifact restores bit-correctly	scheduled restore-and-checksum jobs
RR restore rehearsal	order edges and actions: the node can actually be rebuilt	DR drills, game days [39]
FD failover drill	alternative-route actions engage under load	regional failover exercises
CE chaos experiment	behaviour under injected dependency failure	chaos tooling [7, 16]
CA configuration attestation	referenced recovery assets exist, are pinned and resolvable	CI checks over IaC and runbooks
CT contract/validation test	the O validation predicate itself executes and passes	synthetic transaction suites
CV credential validity	authz edges: recovery credentials work and are reachable out-of-band	secret scanners, break-glass tests

idence liveness: no node violates readiness silently—expired required evidence raises a finding; (**C5**) *acyclicity and seeding*: W3–W4 re-verified on every graph change; (**C6**) *seed externality*: each seed's recovery procedure and credentials are stored and evidenced outside every modelled failure domain (the rule the runbook-in-the-wiki violates). Each rule is checkable in time polynomial in $|V| + |E_{\mathrm{R}}| + |D| + |Ev|$; C1–C2 violations over time constitute *drift*, quantified in Sec. 5. The reference implementation (Sec. 7.6) implements C2, C4, C5, and C6 exactly as stated.

4. QUERYABLE RECOVERY: OPERATIONS AND ALGORITHMS

The model earns the adjective *queryable* by supporting, with standard graph machinery, exactly the questions that Sec. 2.1 showed prose cannot answer. All operations below are implemented in the reference implementation.

4.1 Recovery paths and orders

DEFINITION 13 RECOVERY PATH. *For $v \in V$, the* hard closure $\mathrm{cl}(v)$ *is the set of nodes reachable from v through hard edges of E_{R} in the requires direction (transitive prerequisites). The* recovery path $P(v)$ *is the sub-action-graph of $\mathcal{A}(G_{\mathrm{R}})$ induced by $\mathrm{cl}(v) \cup \{v\}$: everything that must happen, and every ordering constraint among those happenings, for v to reach* O *from total loss. The* recovery path length $\mathrm{RPL}(v)$ *is reported as a pair: $\mathrm{RPL}_{\mathrm{hop}}(v)$, the number of edges on a longest hard chain ending at v, and $\mathrm{RPL}_{\mathrm{time}}(v) = \mathrm{RTO}_{\mathrm{est}}(v)$, its duration-weighted counterpart.*

Computing $\mathrm{cl}(v)$ and $\mathrm{RPL}_{\mathrm{hop}}$ is a reverse reachability and DAG-depth computation, $O(|V| + |E_{\mathrm{R}}|)$; a valid execution order for any target set is a topological sort of the induced action graph [35]; and $\mathrm{RPL}_{\mathrm{time}}$ comes from the critical-path propagation of Prop. 2. Algorithm 1 states the schedule computation concretely; its output is precisely the data behind the timeline of Fig. 7.

4.2 Bottlenecks: blocking centrality

Which node, if unrecoverable, hurts most? Under the hard-edge semantics recovery is *conjunctive*—v needs *all* of $\mathrm{cl}(v)$—so the right necessity notion is membership in the closure, and the right centrality is:

DEFINITION 14 BLOCKING CENTRALITY. $B(x) = \sum_{v\,:\,x \in \mathrm{cl}(v)} w_v$, *the total criticality weight of nodes whose recovery transitively requires x.*

B is computable for all x in $O(|V| \cdot (|V| + |E_{\mathrm{R}}|))$ by reverse reachability, and under conjunctive semantics it coincides exactly with *counterfactual impact*: the weight rendered unrecoverable if x is removed (Sec. 4.3). We note a subtlety that we ourselves initially got wrong: classical dominator analysis [40] is *not* the correct tool here, because dominators characterise unavoidability under *or*-semantics (some path), whereas hard recovery dependencies compose with *and*-semantics (all prerequisites)—under which *every* closure member is unavoidable and the interesting quantity is how much weight sits above it. Dominator analysis becomes appropriate exactly when the model is extended with *alternative recovery routes* (a node recoverable via snapshot restore *or* rebuild-from-source): necessity across alternatives is then dominator membership in the route graph. We flag this and/or extension as future work and keep the base model conjunctive.

Algorithm 1: EARLIESTSCHEDULE(G_{R}) — earliest start/finish and $\mathrm{RTO}_{\mathrm{est}}$ for all nodes.

Input: well-formed G_{R} with durations $d_v(\ell)$
Output: ES, EF over tasks; $\mathrm{RTO}_{\mathrm{est}}(v) = EF[(v, \mathsf{O})]$

```
A ← action graph of G_R (Def. 7);
if A has a cycle then return INFEASIBLE with its strongly connected components [52];
foreach task t = (v, ℓ) in topological order of A do
    ES[t] ← max{EF[p] : p ∈ pred(t)} ∪ {0};
    EF[t] ← ES[t] + d_v(ℓ);
return ES, EF;
```

4.3 Cycles, seeds, and counterfactuals

Circular-bootstrap detection is strongly-connected-component computation on $\mathcal{A}(G_{\mathrm{R}})$ [52]: any non-trivial SCC in the hard core is a P2 finding, reported with its member actions so that engineers can break it (typically by re-homing an asset out of the failure domain, or by demoting an edge to soft with a documented degraded procedure). The level granularity of $\mathcal{A}$ keeps benign cross-level mutual references out of the findings (Sec. 3.5). *Seed audit* checks W4/C6. The *counterfactual operator* $\mathrm{cf}(x)$ removes x and reports the set $\{v : x \in \mathrm{cl}(v)\}$ made unrecoverable and the $\mathrm{RTO}_{\mathrm{est}}$ deltas for the remainder; applied to sets, it answers scenario questions of the form "what if the backup region and the registry are lost together?"

4.4 Mapping to property-graph query languages

G_{R} maps directly onto the property-graph data model [4]: node labels Service, Resource (with seed, tier, weight, durations, and current σ as properties); a relationship type REQUIRES carrying (κ, h, θ, π); evidence as Evidence nodes linked by EVI-

DENCED_BY, carrying class, timestamps, horizon, verdict, and provenance. Every operation of this section then becomes expressible in Cypher [21] or, prospectively, in the ISO GQL standard [32]; closure and ordering use variable-length path patterns, and the metrics of Sec. 5 are aggregation queries. Figure 4 shows a representative governance query—expiring evidence on tier-1 recovery paths—and its result on the worked example: a single near-expired sandbox credential surfaces as a risk to three tier-1 services. The full schema and a query catalogue (recovery order for a target set; unready tier-1 services; confidence bottleneck per service; drift report) appear in B.

5. METRICS FOR RECOVERY POSTURE

Requirements R5–R6 ask for quantities that can be tracked over time, compared across services, and defended in an audit. We define six, chosen so that each is computable in polynomial time from (G_{R}, Ev, D), has explicit bounds and monotonicity behaviour, and—critically—has a documented failure mode. Table 4 summarises; all six are implemented in the reference implementation and instantiated on the worked example in Table 5.

Properties.. All ratio metrics lie in $[0, 1]$. By Prop. 3, RCS, RRI, and REC are monotone non-decreasing under addition of passing evidence and non-increasing under pure passage of time; by Cor. 1, RPL and $\mathrm{RTO}_{\mathrm{est}}$ are monotone non-decreasing under edge addition. This pair of monotonicities yields the suite's central discipline: *structure metrics can only worsen as modelling improves, while evidence metrics can only improve as practice improves*. An organisation that starts modelling honestly should expect $\mathrm{RTO}_{\mathrm{est}}$ estimates to grow and RDC to grow with them—the estimates were always that large; they were merely unknown—and should distrust any dashboard on which $\mathrm{RTO}_{\mathrm{est}}$ shrinks while RDC is flat.

Interpretation and aggregation guidance.. Weighted means mask exactly the failures that matter, so RRI, REC, and RCS are specified to be reported *with* their tier-1 minima and distributions, never as a single scalar; the worked example illustrates why—its weighted RCS is 0.35 while its tier-1 minimum is ≈ 0, and the second number is the finding. Density δ and the recovery-only ratio ρ are *descriptive* statistics of the model, not health scores: their role is anomaly detection (a tier-1 service whose δ-neighbourhood is far sparser than its peers' is more plausibly under-modelled than simple) and quantifying Prop. 1 in the field (RQ2). They must not appear as optimisation targets.

Failure modes and gaming.. Because RCS/REC/RRI are monotone in evidence, the rational way to game them is to manufacture hollow evidence—drills that exercise nothing, attestations rubber-stamped. Three mitigations are part of the model rather than afterthoughts: evidence carries provenance linking to raw records (Def. 10), class requirements are per-node policy set by a governance function rather than by the service owner being measured (Sec. 6.2), and audit sampling re-executes a random subset of evidence-producing procedures. RDC is gameable by classifying every runtime edge *irrelevant*; the countermeasure is that *irrelevant* is itself an attested claim (it asserts the service starts and validates without the dependency), spot-checkable by chaos experiment—which turns the gaming attempt into a testable statement, arguably the best outcome available. We regard the explicitness of these failure modes as a feature of the suite: each metric's blind spot is named, and paired with the mechanism that watches it.

6. REFERENCE ARCHITECTURE AND GOVERNANCE

A model that must be hand-maintained from scratch will not survive contact with an organisation. This section describes how a Recovery Graph is populated from artifacts that already exist, kept consistent, and embedded in governance—the sense in which it is a *governance artifact* and not merely another infrastructure graph. Figure 5 shows the reference architecture.

6.1 Populating the graph

Structure.. Nodes and candidate edges are harvested, not invented. Service inventory comes from the deployment platform and service catalogue [15]; runtime edges D from tracing [49]; resource nodes and order edges from parsing IaC and pipeline definitions (image references imply registry edges; state backends imply repository edges; secret mounts imply authz edges); data edges from backup catalogues and datastore topology. Crucially, harvesting produces *candidates*: the classification γ of each runtime edge and the hardness of each candidate recovery edge are human attestations by the owning team, because they encode counterfactual knowledge ("we can start degraded without recommendations") that no observation of the healthy system contains. The division of labour is deliberate—machines propose, owners attest, and RDC measures how much of the proposal queue has been adjudicated.

Evidence.. Every existing verification practice becomes a producer: scheduled restore tests emit BI/RR items; game days and failover exercises emit RR/FD [39]; chaos experiments emit CE [7, 16]; CI checks over recovery assets emit CA; synthetic-transaction suites emit CT; secret and break-glass scanners emit CV. The integration is thin by design—a webhook per producer appending $(v, c, t, \tau_e, verdict, prov)$—because the model asks producers only for what they already know. The append-only evidence ledger, with provenance, is what distinguishes the graph from a wiki: claims arrive with their substantiation attached, and expire without it.

Lifecycle.. The graph participates in five moments: *design time* (authoring and attestation, in version control, reviewed like code [44]); *run time* (continuous consistency checks C1–C6 and drift detection); *drill time* (plans are executed as exercises; outcomes append evidence and correct durations $d_v(\ell)$); *incident time* (the planner emits the current recovery order and timeline; the console tracks σ against it); and *audit time* (metrics, evidence trails, and query results are exported). The dashed loop in Fig. 5 is the point: a Recovery Graph is not a document that decays but a hypothesis under continuous test.

6.2 The Recovery Graph as a governance artifact

Three governance properties distinguish G_{R} from the infrastructure graphs it superficially resembles.

Separation of claim and assessment. Service owners attest structure (γ, edges, actions); a resilience function—not the owners being measured—sets policy: required evidence classes $req(v)$, validity horizons τ_e, criticality weights, and target levels. This separation is what lets RRI/REC/RCS function as controls rather than self-grades (Sec. 5).

Auditability by construction. Every number the suite produces decomposes into named nodes, attested edges, and provenance-linked evidence items; an auditor can descend from "RRI $= 0.84$" to the three unready nodes to the specific expired credential check behind one of them (Fig. 4). This addresses a concrete regulatory demand: the EU Digital Operational Resilience Act requires finan-

(a) query: expiring evidence on tier-1 recovery paths

```
MATCH (s:Service {tier:1})
      -[:REQUIRES*0..]->(x)
      -[:EVIDENCED_BY]->(e:Evidence {verdict:'pass'})
WHERE e.expiresAt < date() + duration('P30D')
RETURN s.name  AS service,
       x.name  AS atRisk,
       e.class AS evidence,
       e.expiresAt
ORDER BY e.expiresAt
```

(b) result subgraph

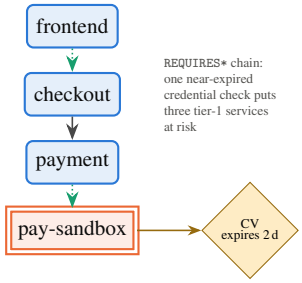

Fig. 4. Queryability in practice: (a) a Cypher query over the property-graph mapping of G_{R}; (b) the result on the worked example of Sec. 7.6. The query is a governance control, not an incident-time tool: it runs continuously, and its non-empty result is a finding.

Table 4. The metric suite. w_v are criticality weights; $W = \sum_v w_v$.

Metric	Definition	Reading
$\mathrm{RPL}(v)$	$\big(\mathrm{RPL}_{\mathrm{hop}}(v),\ \mathrm{RTO}_{\mathrm{est}}(v)\big)$ (Def. 13)	structural and temporal depth of v's recovery
RCS	per-node $C(v)$ (Def. 12); system: distribution, $\frac{1}{W}\sum_v w_v C(v)$, and $\min_{\text{tier-1}} C$	how substantiated recovery claims are
RRI	$\frac{1}{W}\sum_v w_v\,[\mathit{ready}(v)]$	fraction of criticality currently evidence-ready
REC	$\frac{1}{W}\sum_v w_v \frac{\lvert\{c \in \mathit{req}(v) : \text{known-good}\}\rvert}{\lvert \mathit{req}(v)\rvert}$	fineness: how much of the required evidence exists
$\delta,\ \rho$	$\delta = \frac{\lvert E_{\mathrm{R}}\rvert}{\lvert V\rvert(\lvert V\rvert-1)};\ \rho = \frac{\lvert\{e \in E_{\mathrm{R}} : e^{\flat} \notin D \cup D^{-1}\}\rvert}{\lvert E_{\mathrm{R}}\rvert}$	descriptive: modelling density; share of recovery knowledge invisible to tracing
RDC	$\frac{\lvert\{d \in D : \gamma(d) \text{ defined}\}\rvert}{\lvert D\rvert}$	assessment coverage: the trust qualifier for every path/$\mathrm{RTO}_{\mathrm{est}}$ result

cial entities to maintain, test, and demonstrate ICT response and recovery capability [20]; ISO 22301 requires documented business-continuity procedures and evaluation of their effectiveness [30, 31]; NIST SP 800-34 requires contingency plans to be tested and maintained [50]. We claim, precisely, that G_{R} *supports the production of evidence* for such obligations—machine-checkable inventories of recovery dependencies, freshness of testing, and coverage of assessment—and we do not claim that deploying it constitutes compliance with any instrument.

Self-reference, closed. The graph is itself a recovery dependency of the organisation: consulting the plan must not require the systems the plan recovers. Consistency rule C6 therefore applies to the graph platform itself—it must be a seed, with an out-of-band, periodically evidenced access path. The model closes over its own storage; the runbook-in-the-wiki pathology cannot re-enter through the front door.

6.3 Adoption path

Nothing above requires big-bang adoption. The minimal viable graph is tier-1 services plus the platform resources they touch—typically tens of nodes—with RDC measuring, from day one, how much of the runtime edge set has been adjudicated; the metrics are meaningful at any coverage because they are reported with coverage attached (Cor. 1 and Sec. 5). Modelling effort at this granularity is an explicit measurement of RQ1 (Sec. 7), not a promise; the worked example of Sec. 7.6, 19 nodes and 57 edges for an 11-service system, indicates the order of magnitude involved.

7. EVALUATION DESIGN AND FEASIBILITY ILLUSTRATION

We deliberately separate what this paper *demonstrates* from what must be *measured*. The model, algorithms, and metrics are demonstrated: they are formally specified, implemented, and exercised end-to-end on a fully computed example (Sec. 7.6). The empirical claims—that real systems can be modelled at acceptable cost, that recovery structure diverges from runtime structure at scale, that graph-derived plans and estimates predict observed recovery, and that queryability helps humans—are stated here as a registered-style design with hypotheses, baselines, and analysis plans, and no results are reported or implied. We consider this the honest division for a model paper; the design is specific enough to be executed, and to be criticised.

7.1 Research questions and subjects

RQ1 (Expressiveness and cost). Can practising engineers capture the recovery procedures of real cloud-native systems as well-formed Recovery Graphs, and at what modelling cost?

RQ2 (Divergence). How large, in real systems, is the divergence between recovery dependency structure and runtime dependency structure that Prop. 1 establishes in principle?

RQ3 (Predictive utility). Do graph-derived plans, $\mathrm{RTO}_{\mathrm{est}}$ estimates, and blocking-centrality rankings predict observed recovery behaviour under injected failures?

RQ4 (Queryability and governance value). Do operators answer recovery-planning and audit questions faster and more accurately with a queryable Recovery Graph than with conventional artifacts?

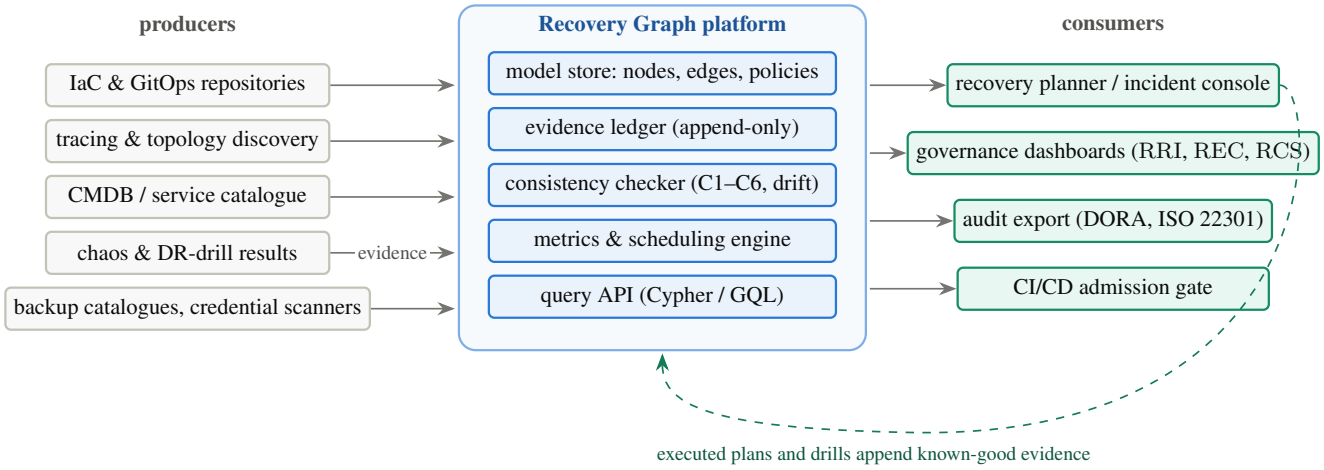

Fig. 5. Reference architecture. Producers feed structure and evidence into the graph platform; consistency checking runs continuously; consumers query. The dashed loop is the system's epistemic engine: executing plans and drills is what refreshes known-good evidence.

Subjects. Three open-source microservice systems spanning an order of magnitude in size: Online Boutique (11 services; Google's microservices demo), the DeathStarBench social network (∼30 services) [23], and TrainTicket (∼41 services), the largest widely used microservice benchmark [61]—each deployed on Kubernetes with the platform resources of Sec. 7.6 (registry, IaC repository, secret store, backup store, DNS, observability, external sandboxes). Two industrial case studies under non-disclosure complement the benchmarks for RQ1–RQ2. Fault injection uses standard chaos tooling [16]; runtime graphs are mined from OpenTelemetry traces under generated load.

7.2 RQ1: modelling feasibility and cost

For each subject, two engineers *independently* author G_{R} from documentation, IaC, deployment manifests, and interviews, using the reference implementation's schema. Measures: person-hours per service; $|V|$, $|E_{\mathrm{R}}|$, RDC achieved; well-formedness findings encountered (cycles, unseeded sources); and *inter-modeller agreement*—per-edge agreement on γ classification (Cohen's κ) and graph edit distance between the two authored graphs, because a model that two competent engineers instantiate very differently is a weak measurement instrument regardless of its formal virtues. Pre-registered expectation: tier-1-scope modelling cost below one person-week per subject; $\kappa \geq 0.6$ on hard/soft/irrelevant classification. Disagreements are themselves data: each is resolved in conference and coded by cause (missing knowledge vs. ambiguous semantics vs. error).

7.3 RQ2: structural divergence

From the traced runtime graph D and the authored E_{R} we compute: the recovery-only ratio ρ; precision/recall of D as a predictor of the hard recovery relation; the number of *inversions* (pairs where runtime direction and recovery order disagree, e.g. queue consumers that must precede producers); and node-set divergence $|R|/|V|$. Pre-registered hypothesis H2: $\rho \geq 0.5$ in every subject—i.e. at least half of recovery knowledge is invisible to runtime observability. H2 is directly falsifiable and its refutation would be a substantive finding against the paper's motivation.

7.4 RQ3: predictive utility under injected failures

Scenarios (per subject): S1 total cluster loss; S2 stateful-store loss with restore from backup; S3 secret/credential-store outage; S4 registry outage; S5 partial loss with constrained capacity (the metastability-prone case). **Conditions**: (B0) *restore-only*: declarative resync of all workloads at once—the level of automation today's platforms provide; (B1) *expert runbook*: recovery executed by an engineer following the subject's written runbook, authored before graph modelling to avoid contamination; (B2) *graph-guided*: recovery executed by following the plan of Algorithm 1. **Measures**: time-to-O per service against probe-defined validation; ordering violations (a service attempting recovery before a hard prerequisite reached its threshold); failed-start and restart counts; occurrence of retry-storm signatures; and, for B2, the prediction error of $\mathrm{RTO}_{\mathrm{est}}$ (absolute percentage error per service) and whether the empirically binding constraint matches the computed critical path and top blocking-centrality nodes. Each (scenario, condition) cell is run ≥ 10 times; analysis uses paired nonparametric tests with effect sizes and confidence intervals; durations $d_v(\ell)$ for prediction are calibrated on runs disjoint from those used for error measurement, and the evidence-decay sensitivity analysis of Sec. 3.6 is performed here. We note explicitly what B2 can and cannot show: an advantage over B0 tests the value of *ordering and gating knowledge*; an advantage over B1 tests the value of its *formalisation*; neither tests authoring cost, which is RQ1's subject.

7.5 RQ4: human queryability and governance value

A within-subject controlled study with $n \geq 16$ professional SREs/-platform engineers. Each participant answers a battery of twelve recovery-planning and audit questions (of the kinds listed in Sec. 4: prerequisite order for a target, unready tier-1 services, best-case recovery time, single points of recovery failure, expiring evidence, counterfactual loss of a resource) under two counterbalanced conditions: (A) the subject's runbooks plus a CMDB-style inventory export; (B) the Recovery Graph console with the query catalogue of B. Measures: time to answer, answer accuracy against gold labels computed from the graph, and NASA-TLX workload; a governance sub-study has compliance practitioners map required DORA/ISO 22301 evidence items to artifacts obtainable in each condi-

tion and rate sufficiency on a rubric. Learning effects are controlled by question-set rotation; the study protocol requires ethics approval and reports demographics and expertise moderators.

7.6 Feasibility illustration: a fully computed example

To demonstrate that the machinery is executable end-to-end—and to fix intuitions with concrete numbers—we model Online Boutique from its public architecture, together with eight platform resources, and compute every quantity in this paper with the reference implementation. The scenario (durations, evidence ages, policies) is constructed and clearly labelled as such: the numbers below are *computed outputs of the model on a declared instance*, not measurements of a production system, and they validate executability, not effectiveness.

The instance has $|V| = 19$ (11 services, 8 resources), $|E_{\mathrm{R}}| = 57$ typed edges, and a traced runtime graph of $|D| = 15$ call edges. Figure 6 shows the graph with computed per-node confidence and readiness; Fig. 7 shows the computed earliest-completion schedule. Table 5 reports the six metrics.

Five observations, each traceable to a figure or table row. *(i)* Three quarters of the recovery edges ($\rho = 0.754$) have no counterpart in the runtime graph—the registry, IaC, backup, secrets, and sandbox dependencies that tracing cannot see—giving a first concrete magnitude for RQ2's hypothesis. *(ii)* The critical path runs through the stateful store and its backup, not through the most-connected service; the largest single contributor to $\mathrm{RTO}_{\mathrm{est}}$ is cluster rebuild followed by data restore (Fig. 7). *(iii)* The evidence scenario contains one nearly expired credential check on the payment sandbox ($\varphi = 0.07$); the weakest-link semantics propagates it to $C(\mathsf{payment}) = 0.02$, $C(\mathsf{checkout}) = 0.01$, $C(\mathsf{frontend}) \approx 0.002$—the model's way of saying that the checkout path's recovery is, today, an assumption resting on a credential nobody has revalidated. The weighted mean $\mathrm{RCS} = 0.353$ conceals this; the mandated tier-1 minimum exposes it (Sec. 5). *(iv)* $\mathrm{RRI} = 0.844$ with exactly three unready nodes (payment, email, paysandbox), each with the missing evidence class identified—the console query of Fig. 4 returns precisely these. *(v)* A pathological variant that stores the cluster-rebuild runbook in an in-cluster wiki is rejected by the consistency checker with the two-node cycle $\{\mathsf{k8s}, \mathsf{wiki}\}$ reported at action granularity—the Meta-2021/runbook-in-the-wiki pattern caught at authoring time rather than at 3 a.m.

7.7 Threats to validity

Construct. Probe-defined O operationalises "operational"; a service can pass probes yet fail users, so RQ3's validation suites must be reviewed by subject-system maintainers. C is an index, not a probability (Remark 1); any reading of RQ3's calibration analysis as risk quantification would exceed the construct. Assumption A1 idealises away mid-recovery regressions; scenario S5 exists to probe where the idealisation breaks. *Internal.* Authors of the model must not author the runbook baseline (B1) or the gold labels for RQ4; independent authorship and blinding are specified above. Graph authoring by paper authors would bias RQ1 favourably; hence external modellers and agreement statistics. Duration calibration and prediction-error measurement use disjoint runs. *External.* Benchmarks are smaller and cleaner than enterprise estates; all subjects are Kubernetes-based; public postmortems over-represent large operators (Sec. 2). The industrial case studies mitigate but do not remove this; claims are scoped to cloud-native systems of the studied kind. *Conclusion.* Chaos runs are high-variance; ≥ 10 repetitions, nonparametric paired analysis, and reported effect sizes with confidence intervals are specified; the user study's n bounds detectable effects, and we commit to reporting confidence intervals rather than binary significance verdicts.

7.8 Reproducibility and artifact availability

The reference implementation (`rgkit`: model, checks, algorithms, metrics; ∼500 lines of Python over `networkx`), the full worked-example instance (nodes, edges, durations, evidence scenario), the pathological variant, and the scripts that generated every number and figure datum in Secs. 7.6 and 5 are publicly available at `https://github.com/obedebessa/recovery-graph`; a self-test verifies the stated formal properties (monotonicity, weakest-link, $\mathrm{RTO}_{\mathrm{est}}$ monotonicity, cycle detection) on the example. The evaluation design's execution assets—subject-system manifests, chaos scenario specifications, probe suites, and study materials—are structured for archival with a DOI upon execution. No proprietary data is required to reproduce anything reported in this paper.

8. RELATED WORK

Table 6 positions the Recovery Graph against nine adjacent bodies of work along the capabilities the requirements of Sec. 2 demand. The short version: each area contributes an ingredient—data protection, testing discipline, dependency structure, declarative reconstruction, governance obligation—and none represents operational recovery knowledge as a queryable, evidence-carrying artifact.

Backup, disaster recovery, and their automation.. Backup and DR tooling protects and re-instantiates *data and infrastructure*; cloud-era work economised it [58] and Keeton et al. automated the *design* of storage DR configurations against cost and objectives [36]. This line optimises what we call the restore prefix, and its objective functions (RPO/RTO of systems) are inputs G_{R} consumes as edge attributes and targets; none of it models cross-service operational recovery, order, or evidence. Kubernetes-native backup tools [57] inherit the same scope.

Business continuity and digital-resilience governance.. ISO 22301 and the BIA guidance of ISO/TS 22317 mandate that organisations know their dependencies, priorities, and recovery procedures, and that these be exercised [30, 31]; NIST SP 800-34 does the same for federal systems [50]; DORA turns testing and demonstrability into binding obligation for a large regulated sector [20]. These instruments define *what must be known*; they are silent on *representation*, and in practice are satisfied with documents. The Recovery Graph is best read as a mechanisation target for exactly this layer—Sec. 6.2 maps its artifacts onto the obligations—which is also why we call it a governance artifact rather than an infrastructure graph.

Chaos engineering and resilience testing.. Chaos engineering builds confidence through experiments in production [7, 47], and Google's DiRT institutionalised disaster testing [39]. This community produces precisely the substantiation our evidence layer consumes, but leaves it unattached: an experiment's result lives in a report, not on the dependency whose recovery it substantiates, and decays without a clock. In our terms, chaos engineering is an evidence *producer* (classes CE, RR, FD) for a store that has not existed. The relationship is symbiotic, not competitive.

Site reliability engineering.. The SRE literature supplies the operational philosophy—error budgets, runbooks, postmortems, the calculus of dependency availability [1, 8, 53]—and empirical incident studies document both the cost of recovery and the decay of prose knowledge [14, 18, 24, 26]. We formalise the artifact this

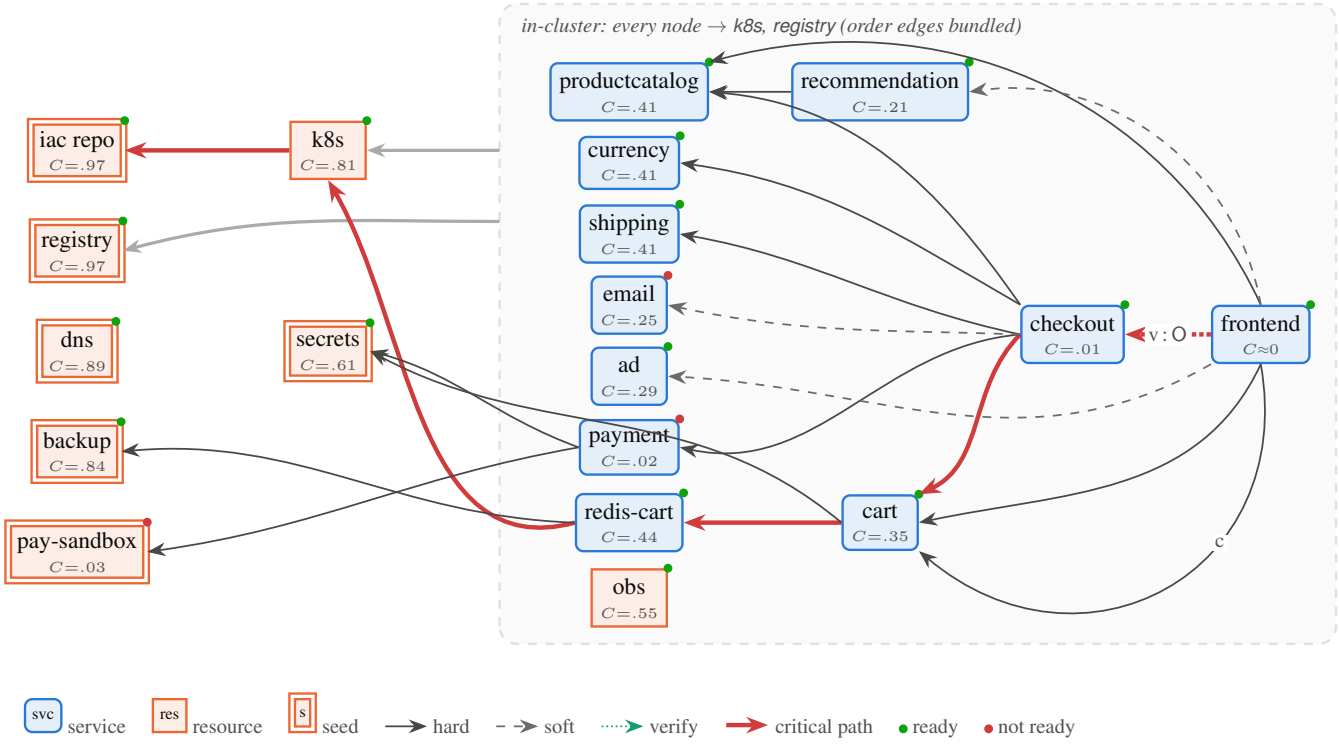

Fig. 6. Recovery Graph of the worked example (Online Boutique services, blue; platform resources, orange; seeds double-bordered). Arrows read "requires"; the red overlay is the computed critical path to frontend. Per-node confidence C and readiness dots are computed from the declared evidence scenario. Four long-range edges and the eleven verify→obs edges are omitted for legibility (full instance in the artifact).

Table 5. Metric suite computed on the worked example (output of `rgkit`).

Metric	Value	Metric	Value
RPL(frontend)	6 hops / 130 min	RRI (weighted)	0.844
RCS weighted mean	0.353	REC (weighted)	0.922
RCS tier-1 minimum	0.002	density δ	0.167
recovery-only ratio ρ	0.754	RDC	0.867

practice keeps informally; the runbook does not disappear but becomes the action documentation attached to nodes, with its executability and freshness now checkable (C3, C4).

Kubernetes, GitOps, and infrastructure-as-code.. Reconciling controllers converge actual toward desired state [5, 11], and Terraform's resource graph orders resource creation and destruction [27, 44]—a true dependency DAG, but over *infrastructure resources*, with no capability levels, no data/verify/capacity semantics, no evidence, and no notion that a running deployment might not be a recovered service. These systems are the reason restore is a solved problem, and their manifests are a harvesting source for G_{R} (Sec. 6.1); the Recovery Graph begins where their semantics end, at $\sigma \geq \mathsf{P}$.

Dependency graphs, configuration graphs, and CMDBs.. Trace-derived service graphs [42, 49] and the RCA systems built on them [22, 60] model the runtime relation D, which Prop. 1 separates from E_{R}; failure-propagation analysis runs along D, recovery along E_{R}, and conflating the two graphs is precisely the error the model is built to prevent. The CMDB is the closest ancestor in spirit—a queryable store of configuration items and relationships [6], with service catalogues as its modern descendant [15]—and its chronic failure mode is instructive: lacking executable semantics or expiring evidence, a CMDB drifts into a stale inventory nobody trusts. G_{R} differs on exactly those two axes (guarded transition semantics; evidence with clocks) plus consistency checking against observed reality (C1–C2), which is what makes drift visible instead of terminal.

Classical recovery, recovery-oriented computing, and self-healing.. Rollback-recovery and distributed snapshots give process-level state restoration its theory [13, 19]; recovery-oriented computing argued for designing systems around fast repair [10, 12, 45]; autonomic computing supplies the MAPE-K loop in which a knowledge component drives self-management [38]. We are complementary to the first (their recovery is a mechanism a data edge may invoke; terminological collision noted—their "recovery line" is a consistent cut, not our recovery path), aligned with the second's philosophy at a different granularity, and, for the third, can

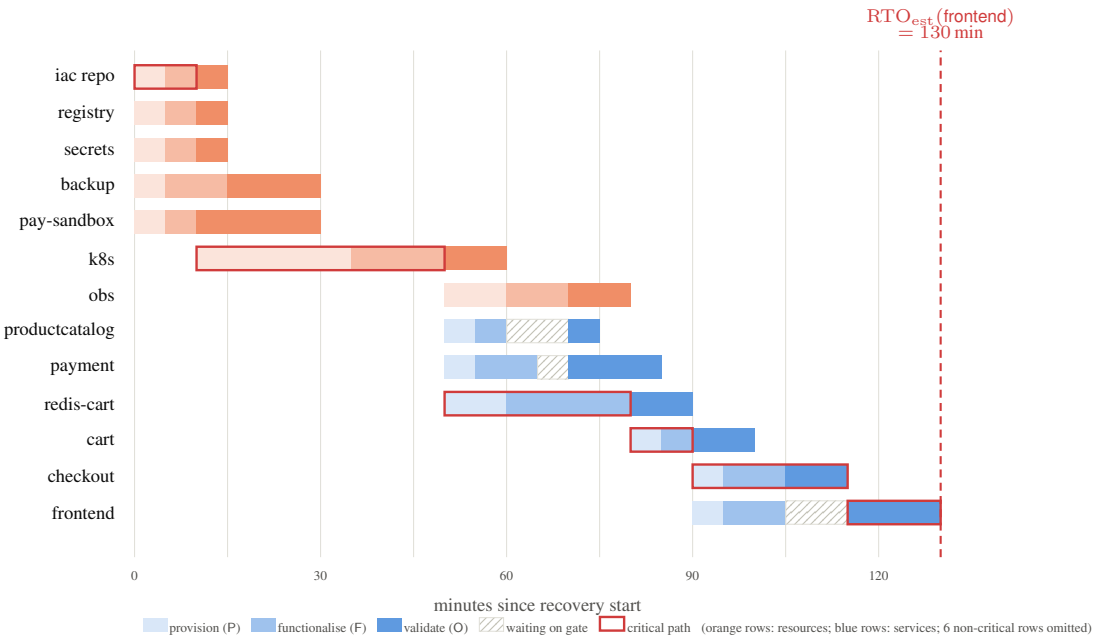

Fig. 7. Computed earliest-completion recovery schedule for the worked example (Algorithm 1; six non-critical rows omitted). Hatched intervals are gate waits—e.g. frontend is functional at 105 min but cannot be certified operational until checkout validates at 115 min. $\mathrm{RTO}_{\mathrm{est}}(\mathsf{frontend}) = 130$ min; the critical chain runs iac → k8s → redis-cart → cart → checkout → frontend, i.e. through the stateful store, not through the largest or most-called service.

Table 6. Positioning. ● = central capability; ◐ = partial/implicit; ○ = absent.

	recovery-specific deps	restore ≠ recover	expiring evidence	computable plans, $\mathrm{RTO}_{\mathrm{est}}$	queryable	governance artifact
backup / DR tooling	◐	○	◐	○	○	○
DR design automation [36]	◐	○	○	◐	○	○
BC standards [20, 30, 50]	◐	◐	◐	○	○	●
chaos engineering [7]	○	◐	◐	○	○	○
SRE practice [8, 39]	◐	◐	◐	○	○	◐
Kubernetes / GitOps [5, 11]	○	○	○	◐	◐	○
IaC resource graphs [27, 44]	◐	○	○	◐	◐	○
CMDB / ITIL, service catalogues [6, 15]	◐	○	○	○	●	◐
runtime dependency graphs, RCA [42, 49, 60]	○	○	○	○	●	○
rollback-recovery protocols [19]	◐	○	○	◐	○	○
ROC / self-healing [38, 45]	○	◐	○	○	○	○
fault trees / attack graphs [48, 56]	○	○	○	◐	◐	◐
digital twins [34, 51]	○	○	○	◐	◐	○
Recovery Graph (this paper)	●	●	●	●	●	●

be read as a concrete proposal for the long-underspecified K: the knowledge model an autonomic recovery manager would need.

Fault trees, attack graphs, and digital twins.. Fault trees and attack graphs demonstrated decades ago that and/or dependency structure over undesired events supports both computation and audit [48, 56]; we adapt that methodological stance to the *return* of capability rather than its loss, with the conjunctive fragment formalised here and the and/or extension noted in Sec. 4.2. Digital twins maintain synchronised virtual replicas for simulation and prediction [34, 51]; a Recovery Graph is deliberately not a twin—it does not simulate behaviour, it represents commitments and their substantiation—but it could serve as the dependency skeleton a resilience-oriented twin animates. Resilience engineering's insistence that resilience is a capability exercised, not a property possessed [28, 59], is the intellectual frame the evidence layer operationalises.

9. DISCUSSION AND LIMITATIONS

Cost, and where the value concentrates.. The model's principal cost is attestation: machines can harvest candidates, but hardness and classification are human judgments, and judgments demand time from the teams with the least of it. Three considerations temper this. First, the marginal cost is lowest exactly where the value is highest—tier-1 services and the platform resources beneath

them—and the metrics remain meaningful at partial coverage because coverage itself is measured (RDC) and estimation error has a known sign (Cor. 1). Second, much of the knowledge is already being produced and discarded: every post-incident review and every drill generates edges and evidence that today evaporate into prose. Third, whether the residual cost is acceptable is not a matter for argument but for RQ1, which measures it. If tier-1 modelling costs an order of magnitude more than our pre-registered expectation, the model as proposed fails its own test, and we would regard that as a result worth publishing.

Staleness, incentives, and gaming.. A Recovery Graph that drifts becomes a CMDB—confidently wrong. The design response is that staleness is itself first-class: evidence expires by construction, drift against inventory and traces is a continuously computed finding, and the metrics reward freshness rather than volume. The gaming analysis of Sec. 5 is deliberately explicit; we do not believe metric suites without named failure modes, and we expect the separation of policy-setting from attestation (Sec. 6.2) to matter more in practice than any formula. What the model cannot manufacture is an organisation that wants to know its recovery posture; it can only make wanting-to-know cheap and not-knowing visible.

Model expressiveness.. Four boundaries of the present formalisation are worth stating plainly. (i) $\mathcal{L}$ is a total order; degraded modes that are incomparable (read-only vs. region-local) need the lattice generalisation, which the mechanisms admit but the paper does not develop. (ii) The base model is conjunctive: alternative recovery routes—restore from snapshot *or* rebuild from source—require and/or structure, at which point necessity analysis becomes dominator-based (Sec. 4.2) and plan choice becomes optimisation. (iii) Durations are deterministic estimates; a stochastic extension (distributions on $d_v(\ell)$, yielding $\mathrm{RTO}_{\mathrm{est}}$ quantiles by simulation over $\mathcal{A}$) is straightforward but its calibration is not, and we prefer honest point estimates paired with coverage over uncalibrated intervals. (iv) Assumption A1 excludes mid-recovery regression; scenario S5 probes this boundary empirically, and a game-semantic treatment (recovery against an adversarial environment) is an intriguing theoretical direction.

Multi-team and multi-cloud scale.. Nothing in the semantics is single-cluster, but federation raises governance questions the paper only gestures at: who attests cross-organisational edges (the payment provider's sandbox), how much of a vendor's recovery posture can be imported as third-party evidence, and how DORA-style obligations on ICT third parties [20] interact with edges that cross company boundaries. We suspect the graph is *more* valuable at these boundaries—they are where tacit knowledge is thinnest—but the claim is untested.

Security of the artifact itself.. A complete map of what must be recovered, in what order, with what credentials, is reconnaissance gold. The graph store therefore warrants the same protection level as the secret store it references, provenance on evidence limits fabrication, and the C6 out-of-band copy must be protected *and* reachable—a genuine tension (availability of the plan vs. confidentiality of the plan) that deployments must resolve consciously, e.g. by tiering what the offline copy contains.

When not to use this.. A five-service monolith with one database does not need a Recovery Graph; it needs a tested restore script and a printed runbook—which is to say, its recovery graph fits in a head. The model earns its overhead where no single head holds the closure: many teams, many stateful systems, platform layers with their own recovery structure, and auditors who ask questions that today take weeks of archaeology. Below that threshold, the checklist is the better technology.

10. CONCLUSION

Cloud-native engineering solved restoration and called it recovery. The distinction between the two—long understood by practitioners, long absent from our formal vocabulary—turns out to be exactly the kind of gap a model can close: what separates a restored system from a recovered service is knowledge (recovery-specific dependencies, order, gates, credentials, validation criteria) and substantiation (whether anyone has recently proved any of it works), and both are representable.

The Recovery Graph represents them: a typed graph over services and recovery resources whose edges carry recovery semantics rather than runtime observations, whose vertices move through capability levels that make *restored* a formal waypoint rather than a triumphant endpoint, and whose claims carry expiring, provenance-linked evidence. From this structure follow computable recovery paths and schedules with lower-bound guarantees, bottleneck and counterfactual analyses, circular-bootstrap detection at authoring time, six metrics with stated properties and stated failure modes, and a direct embedding into standard graph query languages. The computed feasibility illustration shows the machinery working end-to-end—and, in passing, shows three quarters of a real architecture's recovery dependencies to be invisible to runtime observability, a single stale credential collapsing the substantiated confidence of an entire tier-1 chain, and a bootstrap cycle caught by a consistency check instead of an outage.

What the paper does not show is effect: whether real organisations can afford the modelling, how far recovery structure diverges from runtime structure in the wild, whether graph-derived plans beat expert runbooks, and whether queryability changes what operators and auditors can do. These are exactly the four questions the registered evaluation design and the open artifact are built to answer, and we invite the community to answer them adversarially. If the Recovery Graph survives that contact, operational continuity gains what infrastructure gained a decade ago: a declarative, versioned, testable representation of intent—this time, for the way back up.

References

[1] Heather Adkins, Betsy Beyer, Paul Blankinship, Piotr Lewandowski, Ana Oprea, and Adam Stubblefield. *Building Secure and Reliable Systems*. O'Reilly Media, 2020.

[2] Amazon Web Services. Summary of the Amazon Kinesis event in the Northern Virginia (US-EAST-1) region, November 25. `https://aws.amazon.com/message/11201/`, 2020.

[3] Amazon Web Services. Summary of the AWS service event in the Northern Virginia (US-EAST-1) region, December 7. `https://aws.amazon.com/message/12721/`, 2021.

[4] Renzo Angles, Marcelo Arenas, Pablo Barceló, Aidan Hogan, Juan Reutter, and Domagoj Vrgoč. Foundations of modern query languages for graph databases. *ACM Computing Surveys*, 50(5):68:1–68:40, 2017.

[5] Argo Project. Argo CD: Declarative GitOps continuous delivery for Kubernetes. `https://argo-cd.readthedocs.io`. Accessed 2026-08.

[6] AXELOS. *ITIL Foundation: ITIL 4 Edition*. The Stationery Office, 2019.

[7] Ali Basiri, Niosha Behnam, Ruud de Rooij, Lorin Hochstein, Luke Kosewski, Justin Reynolds, and Casey Rosenthal. Chaos engineering. *IEEE Software*, 33(3):35–41, 2016.

[8] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy, editors. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.

[9] Nathan Bronson, Abutalib Aghayev, Aleksey Charapko, and Timothy Zhu. Metastable failures in distributed systems. In *Proc. Workshop on Hot Topics in Operating Systems (HotOS)*, pages 221–227, 2021.

[10] Aaron B. Brown and David A. Patterson. Undo for operators: Building an undoable e-mail store. In *Proc. USENIX Annual Technical Conference*, pages 1–14, 2003.

[11] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, Omega, and Kubernetes. *Communications of the ACM*, 59(5):50–57, 2016.

[12] George Candea, Shinichi Kawamoto, Yuichi Fujiki, Greg Friedman, and Armando Fox. Microreboot — a technique for cheap recovery. In *Proc. 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 31–44, 2004.

[13] K. Mani Chandy and Leslie Lamport. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems*, 3(1):63–75, 1985.

[14] Zhuangbin Chen, Yu Kang, Liqun Li, Xu Zhang, Hongyu Zhang, Hui Xu, Yangfan Zhou, Li Yang, Jeffrey Sun, Zhangwei Xu, Yingnong Dang, Feng Gao, Pu Zhao, Bo Zhang, Qingwei Lin, Minghua Ma, and Michael R. Lyu. Towards intelligent incident management: Why we need it and how we make it. In *Proc. 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), Industry Track*, pages 1487–1497, 2020.

[15] Cloud Native Computing Foundation. Backstage: An open platform for building developer portals. `https://backstage.io`. Accessed 2026-08.

[16] Cloud Native Computing Foundation. Litmuschaos: Open source chaos engineering platform. `https://litmuschaos.io`. Accessed 2026-08.

[17] CrowdStrike. External technical root cause analysis — Channel File 291. `https://www.crowdstrike.com/falcon-content-update-remediation-and-guidance-hub/`, 2024. Published August 2024.

[18] Yingnong Dang, Qingwei Lin, and Peng Huang. AIOps: Real-world challenges and research innovations. In *Proc. 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, pages 4–5, 2019.

[19] E. N. Elnozahy, Lorenzo Alvisi, Yi-Min Wang, and David B. Johnson. A survey of rollback-recovery protocols in message-passing systems. *ACM Computing Surveys*, 34(3):375–408, 2002.

[20] European Union. Regulation (EU) 2022/2554 of the European Parliament and of the Council on digital operational resilience for the financial sector (DORA). Official Journal of the European Union, L 333, 2022. Applicable from 17 January 2025.

[21] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindaaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. Cypher: An evolving query language for property graphs. In *Proc. ACM SIGMOD International Conference on Management of Data*, pages 1433–1445, 2018.

[22] Yu Gan, Mingyu Liang, Sundar Dev, David Lo, and Christina Delimitrou. Sage: Practical and scalable ML-driven performance debugging in microservices. In *Proc. 26th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 135–151, 2021.

[23] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, Kelvin Hu, Meghna Pancholi, Yuan He, Brett Clancy, Chris Colen, Fukang Wen, Catherine Leung, Siyuan Wang, Leon Zaruvinsky, Mateo Espinosa, Rick Lin, Zhongling Liu, Jake Padilla, and Christina Delimitrou. An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In *Proc. 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 3–18, 2019.

[24] Supriyo Ghosh, Manish Shetty, Chetan Bansal, and Suman Nath. How to fight production incidents? An empirical study on a large-scale cloud service. In *Proc. 13th ACM Symposium on Cloud Computing (SoCC)*, pages 126–141, 2022.

[25] GitLab. Postmortem of database outage of January 31. `https://about.gitlab.com/blog/postmortem-of-database-outage-of-january-31/`, 2017.

[26] Haryadi S. Gunawi, Mingzhe Hao, Riza O. Suminto, Agung Laksono, Anang D. Satria, Jeffry Adityatama, and Kurnia J. Eliazar. Why does the cloud stop computing? Lessons from hundreds of service outages. In *Proc. 7th ACM Symposium on Cloud Computing (SoCC)*, pages 1–16, 2016.

[27] HashiCorp. Terraform internals: Resource graph. `https://developer.hashicorp.com/terraform/internals/graph`. Accessed 2026-08.

[28] Erik Hollnagel, David D. Woods, and Nancy Leveson, editors. *Resilience Engineering: Concepts and Precepts*. Ashgate, 2006.

[29] Lexiang Huang, Matthew Magnusson, Abishek Bangalore Muralikrishna, Salman Estyak, Rebecca Isaacs, Abutalib Aghayev, Timothy Zhu, and Aleksey Charapko. Metastable failures in the wild. In *Proc. 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 73–90, 2022.

[30] ISO. ISO 22301:2019 — security and resilience — business continuity management systems — requirements. International Organization for Standardization, 2019.

[31] ISO. ISO/TS 22317:2021 — security and resilience — business continuity management systems — guidelines for business impact analysis. International Organization for Standardization, 2021.

[32] ISO/IEC. ISO/IEC 39075:2024 — information technology — database languages — GQL. International Organization for Standardization, 2024.

[33] Santosh Janardhan. More details about the October 4 outage. Meta Engineering Blog, `https://engineering.fb.com/2021/10/05/networking-traffic/outage-details/`, 2021.

[34] David Jones, Chris Snider, Aydin Nassehi, Jason Yon, and Ben Hicks. Characterising the digital twin: A systematic literature review. *CIRP Journal of Manufacturing Science and Technology*, 29:36–52, 2020.

[35] Arthur B. Kahn. Topological sorting of large networks. *Communications of the ACM*, 5(11):558–562, 1962.

[36] Kimberly Keeton, Cipriano Santos, Dirk Beyer, Jeffrey Chase, and John Wilkes. Designing for disasters. In *Proc. 3rd USENIX Conference on File and Storage Technologies (FAST)*, pages 59–72, 2004.

[37] James E. Kelley and Morgan R. Walker. Critical-path planning and scheduling. In *Proc. Eastern Joint Computer Conference*, pages 160–173, 1959.

[38] Jeffrey O. Kephart and David M. Chess. The vision of autonomic computing. *IEEE Computer*, 36(1):41–50, 2003.

[39] Kripa Krishnan. Weathering the unexpected. *Communications of the ACM*, 55(11):48–52, 2012.

[40] Thomas Lengauer and Robert Endre Tarjan. A fast algorithm for finding dominators in a flowgraph. *ACM Transactions on Programming Languages and Systems*, 1(1):121–141, 1979.

[41] Haopeng Liu, Shan Lu, Madan Musuvathi, and Suman Nath. What bugs cause production cloud incidents? In *Proc. Workshop on Hot Topics in Operating Systems (HotOS)*, pages 155–162, 2019.

[42] Shutian Luo, Huanle Xu, Chengzhi Lu, Kejiang Ye, Guoyao Xu, Liping Zhang, Yu Ding, Jian He, and Chengzhong Xu. Characterizing microservice dependency and performance: Alibaba trace analysis. In *Proc. 12th ACM Symposium on Cloud Computing (SoCC)*, pages 412–426, 2021.

[43] Microsoft. KB5042429: New recovery tool to help with CrowdStrike issue impacting Windows devices. `https://support.microsoft.com/en-us/topic/kb5042429`, 2024.

[44] Kief Morris. *Infrastructure as Code: Dynamic Systems for the Cloud Age*. O'Reilly Media, 2nd edition, 2020.

[45] David Patterson, Aaron Brown, Pete Broadwell, George Candea, Mike Chen, James Cutler, Patricia Enriquez, Armando Fox, Emre Kıcıman, Matthew Merzbacher, David Oppenheimer, Naveen Sastry, William Tetzlaff, Jonathan Traupman, and Noah Treuhaft. Recovery-oriented computing (ROC): Motivation, definition, techniques, and case studies. Technical Report UCB//CSD-02-1175, UC Berkeley, 2002.

[46] Mathieu Rosemain. Millions of websites offline after fire at French cloud services firm OVHcloud. Reuters, 10 March, 2021.

[47] Casey Rosenthal and Nora Jones. *Chaos Engineering: System Resiliency in Practice*. O'Reilly Media, 2020.

[48] Oleg Sheyner, Joshua Haines, Somesh Jha, Richard Lippmann, and Jeannette M. Wing. Automated generation and analysis of attack graphs. In *Proc. IEEE Symposium on Security and Privacy*, pages 273–284, 2002.

[49] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. Dapper, a large-scale distributed systems tracing infrastructure. Technical Report dapper-2010-1, Google, Inc., 2010.

[50] Marianne Swanson, Pauline Bowen, Amy Wohl Phillips, Dean Gallup, and David Lynes. Contingency planning guide for federal information systems. Technical Report NIST Special Publication 800-34 Rev. 1, National Institute of Standards and Technology, 2010.

[51] Fei Tao, He Zhang, Ang Liu, and A. Y. C. Nee. Digital twin in industry: State-of-the-art. *IEEE Transactions on Industrial Informatics*, 15(4):2405–2415, 2019.

[52] Robert Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972.

[53] Ben Treynor, Mike Dahlin, Vivek Rau, and Betsy Beyer. The calculus of service availability. *Communications of the ACM*, 60(9):42–47, 2017.

[54] Jeffrey D. Ullman. NP-complete scheduling problems. *Journal of Computer and System Sciences*, 10(3):384–393, 1975.

[55] Uptime Institute. Annual outage analysis 2024. Uptime Institute Intelligence Report, 2024.

[56] William E. Vesely, Francine F. Goldberg, Norman H. Roberts, and David F. Haasl. Fault tree handbook. Technical Report NUREG-0492, U.S. Nuclear Regulatory Commission, 1981.

[57] VMware Tanzu. Velero: Backup and migrate Kubernetes applications. `https://velero.io`. Accessed 2026-08.

[58] Timothy Wood, Emmanuel Cecchet, K. K. Ramakrishnan, Prashant Shenoy, Jacobus van der Merwe, and Arun Venkataramani. Disaster recovery as a cloud service: Economic benefits and deployment challenges. In *Proc. 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*, 2010.

[59] David D. Woods. Four concepts for resilience and the implications for the future of resilience engineering. *Reliability Engineering & System Safety*, 141:5–9, 2015.

[60] Li Wu, Johan Tordsson, Erik Elmroth, and Odej Kao. MicroRCA: Root cause localization of performance issues in microservices. In *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, pages 1–9, 2020.

[61] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study. *IEEE Transactions on Software Engineering*, 47(2):243–260, 2021.

APPENDIX

A. PROOFS

Theorem 9 (modelled recoverability). Setting. G_{R} satisfies W1–W2; $\sigma_0 \equiv \mathsf{D}$; runs obey Assumption A1 (levels never regress during a run).
($\Leftarrow$) Assume $\mathcal{A}(G_{\mathrm{R}})$ acyclic and every source task owned by a seed. Fix a topological order $t_1, \ldots, t_m$ of $\mathcal{A}(G_{\mathrm{R}})$ and fire actions in that order. We show by induction on i that t_i's guard holds when fired. Every guard atom of $t_i = (v, \ell)$ is of the form $\sigma(x) \geq \pi$ and corresponds to an in-edge $(x, \pi) \to (v, \ell)$ of $\mathcal{A}(G_{\mathrm{R}})$ (Def. 7), whose source precedes t_i in the order; by the induction hypothesis (x, π) has fired, so $\sigma(x) \geq \pi$ held at that point, and by A1 it still holds. If t_i is a source task it has no cross-node atoms; W1 gives an executable action and seed membership (hypothesis (ii)) discharges its external means. Hence the full sequence is a run reaching $\sigma \equiv \mathsf{O}$, and its restriction to $\mathrm{cl}(v) \cup \{v\}$ is a plan for any v.
($\Rightarrow$) Contrapositive, two cases. *(a)* If $\mathcal{A}(G_{\mathrm{R}})$ has a cycle $t_{i_1} \to \cdots \to t_{i_k} \to t_{i_1}$, consider any run and the first member of the cycle it fires, say t_{i_j}: its guard contains the atom contributed by the edge from $t_{i_{j-1}}$ (indices mod k), which has not fired, and under A1 no other mechanism establishes a level lower-bound; so no member fires first, and no node whose closure meets the cycle reaches O. *(b)* If some source task (x, π) belongs to a non-seed x, then x has no in-graph prerequisites and, by definition of B, no external recovery

means; no run advances x, and every node with x in its closure is blocked. □

Proposition 3 (weakest link). Bounds: $\varphi \in [0,1]$ implies $E_{\text{loc}} \in [0,1]$; the recursion multiplies values in $[0,1]$, so $C(v) \in [0,1]$. Domination: for any hard prerequisite x of v, $C(v) = E_{\text{loc}}(v) \cdot \min_{y \in \text{pre}_{\text{h}}(v)} C(y) \cdot \Delta_s \leq \min_y C(y) \leq C(x)$; iterating along hard chains gives $C(v) \leq \min_{x \in \text{cl}(v)} C(x)$. Evidence monotonicity: adding a passing item can only increase some $\max_e \varphi(e)$, hence E_{loc}, and the recursion is monotone in each argument (product of non-decreasing maps; min is monotone); time monotonicity is the same argument applied to φ non-increasing in age. □

Proposition 2. The unbounded-parallelism claim is the classical PERT/CPM longest-path argument over the DAG $\mathcal{A}(G_{\text{R}})$ [37]: earliest start of a task is the maximum earliest finish of its predecessors, computable in one topological pass; optimality follows because every schedule must respect every precedence, so no task can start earlier than the longest weighted chain into it. NP-hardness with k bounded executors: taking all thresholds hard/start, unit durations for (v, P) and zero for the others embeds precedence-constrained scheduling $P \,|\, prec \,|\, C_{\max}$, proved NP-hard by Ullman [54]. □

B. PROPERTY-GRAPH SCHEMA AND QUERY CATALOGUE

Schema. Node labels: Service, Resource — properties `name`, `tier`, `weight`, `seed`, `sigma`, `durP/durF/durO`, `reqClasses`. Relationship REQUIRES — properties `kind` ∈ {`order,data,verify,capacity,authz`}, `hard`, `stage` ∈ {`start,complete`}, `plevel`. Node label Evidence — properties `class`, `producedAt`, `expiresAt`, `verdict`, `provenance`; relationship EVIDENCED_BY from subject to evidence.

Q1: everything that must precede recovery of a target, with order.

```
MATCH p = (t:Service {name:$target})-[:
    REQUIRES* {hard:true}]->(x)
RETURN DISTINCT x.name, max(length(p)) AS
    depth ORDER BY depth DESC
```

Q2: tier-1 services that are not evidence-ready.

```
MATCH (s:Service {tier:1})
WHERE any(c IN s.reqClasses WHERE NOT EXISTS
    {
  MATCH (s)-[:EVIDENCED_BY]->(e:Evidence {
      class:c, verdict:'pass'})
  WHERE e.expiresAt >= date() })
RETURN s.name
```

Q3: confidence bottleneck - the least-substantiated node on each tier-1 closure.

```
MATCH (s:Service {tier:1})-[:REQUIRES* {hard:
    true}]->(x)
WITH s, x ORDER BY x.confidence ASC
RETURN s.name, collect(x.name)[0] AS
    bottleneck, collect(x.confidence)[0] AS c
```

Q4: counterfactual - weight made unrecoverable if r is lost.

```
MATCH (v)-[:REQUIRES* {hard:true}]->(r {name:
    $resource})
RETURN sum(v.weight) AS blockedWeight,
    collect(v.name) AS blocked
```

(`x.confidence` denotes the materialised $C(x)$, recomputed on evidence or structure change; Q1's depth ordering realises the topological presentation of Sec. 4.)

C. WORKED-EXAMPLE INSTANCE DATA

Table 7 lists the per-node computed values behind Figs. 6–7 and Table 5; the full edge list (57 edges), the evidence scenario (35 items), the assumed durations, and the generating script are in the artifact. Durations are labelled assumptions of the illustration scenario, not measurements.

Table 7. Per-node computed values (output of `rgkit`; tier-1 and platform rows shown, alphabetical within group).

node	tier	ready	E_{loc}	C	RPL_{hop}	RTO_{est}
cart	1	yes	0.79	0.35	4	1 h 40
checkout	1	yes	0.63	0.01	5	1 h 55
frontend	1	yes	0.73	0.002	6	2 h 10
payment	1	**no**	0.46	0.02	3	1 h 25
redis-cart	1	yes	0.80	0.44	3	1 h 30
backup	res	yes	0.84	0.84	0	30 m
iac repo	res	yes	0.97	0.97	0	15 m
k8s	res	yes	0.84	0.81	1	1 h 00
obs	res	yes	0.68	0.55	2	1 h 20
pay-sandbox	res	**no**	0.03	0.03	0	30 m
registry	res	yes	0.97	0.97	0	15 m
secrets	res	yes	0.61	0.61	0	15 m